

LANGCHAIN

Langchain is an open-source framework for developing application powered by Large Language Model (LLM)

★ What can you build.

- Conversational Chatbots
- AI Knowledge Assistants
- AI Agents
- Workflow Automation
- Summarization / Research Helpers

★ Langchain Components

- Models
- Prompts
- Chains
- Memory
- Indexes
- Agents

★ Models

In Langchain, "models" are the core interfaces through which you interact with AI models.

- ① Language model $\rightarrow$ text $\rightarrow$ text
- ② Embedding model $\rightarrow$ text $\rightarrow$ vector

★ Prompts

The query which you want to ask LLM

① Dynamic & Reusable Prompts

from langchain_core.prompts import PromptTemplate

```
prompt = PromptTemplate.from_template('Summarise {topic} in {emotion} tone')  
print(prompt.format(topic='Cricket', length emotion='fun'))
```

② Role Based Prompts

```
chat_prompt = ChatPromptTemplate.from_template([  
    ("system", "Hi you are a experienced {profession}"),  
    ("user", "Tell me about {topic}"),  
)
```

```
print(chat_prompt.invoke(profession='engineer', topic='LLM'))
```

★ Chains

Chains is the concept to build the pipeline for this input ~~output~~ ^{output}.
outcome becomes the future component input.

★ Indexes

Indexes connect your application to external knowledge - such as PDFs, websites or database.

- Doc loader
- Text splitter
- Vector store
- Retrievers

LLM + external knowledge

★ Memory

LLM API calls are stateless

- Conversation Buffer Memory
- Conversation Buffer Window Memory
- Summarizer-Based memory
- Custom memory

★ Agents

AI Agents

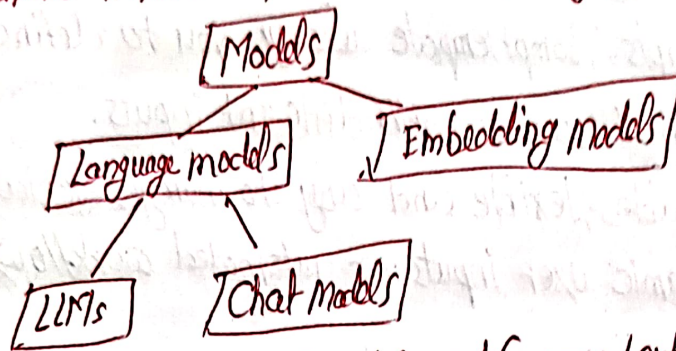
+ LLM (NLU + Text gen)

Chatbox + workers

Reasoning capability + tools

Models

The model component in Langchain is crucial part of framework, designed to facilitate interactions with various language models and embedding models



LLMs:- General-purpose model that is used for raw text generation. They take a string (or plain text) as input and returns a string (plain text). These are traditionally older models and are not used much now.

Chat models:- Language model that are specialized for conversational task. They take a sequence of msg as inputs and return chat msg as output. These are traditionally newer models and used more than LLMs.

LLMs

```
from langchain_openai import OpenAI
from dotenv import load_dotenv
load_dotenv()
llm = OpenAI(model='gpt-3.5-turbo-instruct')
result = llm.invoke("What is the capital of India")
print(result)
```

Chat model

```
from langchain_openai import ChatOpenAI
from dotenv import load_dotenv
load_dotenv()
model = ChatOpenAI(model='gpt-4', temperature=0)
result = model.invoke("Suggest me 5 Indian male names")
print(result.content)
```

Prompt Template

A prompt Template in Langchain is a structured way to create prompts dynamically by inserting variables into a predefined template. Instead of hardcoding prompts, PromptTemplate allows you to define placeholders that can be filled in at runtime with different inputs. This makes it reusable, flexible and easy to manage, especially when working with dynamic user inputs or automated workflows.

Why used PromptTemplate over f strings?

- ① Default validation
- ② Reusable
- ③ Langchain Ecosystem.

Message placeholder

A msg placeholder in Langchain is special placeholder used inside a chat promptTemplate to dynamically insert chat history of a msg at runtime.

Structured Output

In longchain structured output refers to the practice of having language models return responses in a well defined data format (json) rather than free from text. This makes the model output easier to parse and work with programmatically.

Why do we need structured output
Data extraction. API Building

Agents

[With-structured-output] → default structured output give output into json format.

Type dict - Type dict is the way to define a dictionary in python where you specify what key and value should exist. It helps ensure that your dict follow a specific structure.

Code →

```
from typing import TypedDict
class Person(TypedDict):
    name: str
    age: int
new_person: Person = {'name': 'sid', 'age': 35}
print(new_person)
```

Annotation - It is use of the make dout less.

```
name: Annotated[str, "The name of the person"]
age: Annotated[int, "The age of the person"]
```

Pydantic - Pydantic is data validation and data parsing library for Python. It ensures that the data you work with is correct structured and type-safe.

Field import

from pydantic import BaseModel

from pydantic import BaseModel

class Student(BaseModel):

name: str

new_student = {'name': 'sid'}

student = Student(**new_student)

print(student)

print(type(student))

Emails to import
for checking email
are correct or
not

Optional import
for
age: Optional[int] = 14

Annoted replace par
field me description
Kya use krte hai

Output Parsers

Output parsers in Langchain help convert raw LLM responses into structured formats like JSON, CSV, Pydantic models and more. They ensure consistency, validation and ease of use in applications.

- ① String Output Parser
- ② JSON Output Parser
- ③ Structured Output Parser
- ④ Pydantic Output Parser.

StrOutputParser - This is the smallest output parser in Langchain. It is used to parse the output of a LLM and return it as a plain string.

Code ⇒

```
parser = StrOutputParser()
chain = tem1 | model | parser | tem2 | model | parser
chain.invoke({'topic': 'black hole'})
present(result)
```

JSONOutputParser - It is used to output in JSON format.

Structured Output Parser - Structured output parser is an output parser in Langchain that helps extract structured JSON data from LLM responses based on predefined field schemas.

It works by defining a list of fields that the model should return ensuring the output follows a structured format.

Pydantic Output Parser - It is a structured output parser in Langchain that uses Pydantic models to enforce schema validation when processing LLM responses.

Chains

- ① sequential chain ② Parallel chain ③ Conditional chain.

Sequential chain ->

```
prompt = PromptTemplate(
    template = 'Generate 5 line on {topic}',
    input_variables = ['topic']
)
model = ChatOpenAI()
parser = StrOutputParser()
chain = prompt | model | parser
result = chain.invoke({'topic': 'cricket'})
print(result.content)
```

Parallel chain ->

```
ParallelChain = RunnableParallel(
    'notes': prompt1 | model1 | parser,
    'quiz': prompt2 | model2 | parser
)
merge_chain = prompt3 | model3 | parser
chain = ParallelChain | merge_chain
```

Conditional chain ->

```
ClassifierChain = prompt1 | model | parser2
branch_chain = RunnableBranch(
    (lambda x: x.sentiment == 'positive', prompt2 | model | parser),
    (lambda x: x.sentiment == 'negative', prompt3 | model | parser),
    RunnableLambda(lambda x: "Could not find sentiment") -> default
)
```

Runnable

Task Specific runnables - These are core langchain components that have been converted into Runnable so they can be used in pipeline.

Ex: ChatOpenAI
PromptTemplate
Retriever

All type of task like - Base model
Prompt template, Output parsers, Retrievers
tools

Runnable Primitives - These are fundamental building block for structure execution logic in AI workflow.

They help orchestrate execution by defining Runnable interact (sequentially, parallel, conditional)

Ex: RunnableSequence
RunnableMap
RunnableLambda

RunnableSequence - RunnableSequence is a sequential chain of runnables in langchain that executes each step one after another, passing the output of one step as the input to the next.

- It is useful when you need to compose multiple runnables together in a structured workflow.

```
RunnableSequence(prompt1, model, parser, prompt2, model, parser)
```

RunnableParallel - RunnableParallel is a runnable primitive that allow multiple runnables to execute in parallel.

Each runnable receives the same input and processes it independently producing a dictionary of outputs.

```
ParallelChain = RunnableParallel({  
    'tweet': RunnableSequence(prompt1, model, parser),  
    'linkedin': RunnableSequence(prompt2, model, parser)  
})
```

③ RunnablePassthrough - RunnablePassthrough is a special runnable primitive that simply returns the input as output without modifying it.

2 → chain → 2

```
joke_gen_chain = RunnableSequence(prompt1, model, parser)
parallel_chain = RunnableParallel({
    'joke': RunnablePassthrough(),
    'explain': RunnableSequence(prompt2, model, parser)
})
final_chain = RunnableSequence(joke_gen_chain, parallel_chain)
```

④ RunnableLambda - It is a runnable primitive that allow you to apply custom python functions with an AI pipeline.

```
'word_count': RunnableLambda(lambda x: len(x.split()))
```

⑤ RunnableBranch - RunnableBranch is a control flow component in langchain that allow you to conditionally route input data to different chain or runables based on custom logic.

```
short_chain = RunnableSequence(prompt1, model, parser)
long_chain = RunnableSequence(prompt2, model, parser)
branch = RunnableBranch(
    (lambda x: len(x.split()) > 500, long_chain),
    short_chain
)
```

Document loaders

- ① Textloader
- ② Pyloader
- ③ WebBaseLoader
- ④ CSV loader.

Document loaders are components in Langchain use to load data from various sources into a standardized format which can then be used for chunking, embedding, retrieval and generation.

Document {

page_content = 'The actual text content',

metadata = {'source': 'filename.pdf' ... }

}

Textloader - Textloader is a simple and commonly used document loader in Langchain that read plain text (.txt) file and convert them into Langchain Document object.

```
loader = TextLoader('file.txt', encoding = 'utf-8')
```

```
doc = loader.load()
```

Pyloader - Pyloader is document loader in Langchain used to load content from PDF files and convert each page into a Document Object.

```
loader = PyPDFLoader('file.pdf')
```

```
doc = loader.load()
```

Directory loader - Directory loader is a document loader that lets you load multiple documents from a directory (folder) of files

```
loader = DirectoryLoader(
```

```
path = 'books'
```

```
glob = '*.pdf',
```

```
loader_cls = PyPDFLoader
```

```
)
```

from langchain.document_loaders import PyPDFLoader

Web Base loader -> used to extract text content from the web pages (URLs)

It uses BeautifulSoup under the hood to parse HTML and extract visible text.

from langchain_community.document_loaders import YoutubeLoader

loader = WebBaseLoader(url)

import InstaLoader

CSV loader -> CSV loader is a document loader used to load CSV files into Langchain Document objects one per row by default.

```
loader = CSVLoader(file_path='file.csv')  
docs = loader.load()
```

Text splitting

Text splitting is the process of breaking large chunks of text (like articles, PDFs, HTML pages or books) into smaller, manageable pieces (chunks) that an LLM can handle effectively.

Overcoming model limitations -> Many embedding models and language models have maximum input size constraints. Splitting allow us to process documents that would otherwise exceed those limits.

- ① Length Based
- ② Text structure based
- ③ Document Structure base
- ④ semantic & Meaning based.

Length Based Text Splitting -> from langchain.textsplitter import CharacterTextSplitter.

```
splitter = CharacterTextSplitter(  
    chunksize=100,  
    chunkoverlap=0,  
    separator=''  
)  
splitter.split_text(text)
```

HTML Text Splitter use for web pages.

② Text-Structural Based

```
splitter = RecursiveCharacterTextSplitter(  
    chunk_size=300,  
    chunk_overlap=0,  
)  
chunks = splitter.split_text(text)
```

③ Document-Structural Based :-

```
splitter = RecursiveCharacterTextSplitter.from_language(  
    language=Language.PYTHON,  
    chunk_size=300,  
    chunk_overlap=0;  
)
```

④ Semantic Meaning Based :-

```
text_splitter = SentenceChunker(  
    OpenAIEmbedding(), breakpoint_threshold_type="standard-deviation",  
    breakpoint_threshold_amount=1  
)
```

Vector Stores

A vector store is a system designed to store and retrieve data represented as numerical vectors.

Key Features

- ① Storage :- Ensure that vectors and their associated metadata are retained whether in-memory for quick lookups or on disk for durability and large scale use
- ② Similarity Search :- Helps retrieve the vectors most similar to a query vector
- ③ Indexing :- Provide a data structure or method that enables fast similarity searches on high dimensional vectors.
- ④ CRUD operation :- Manage the life cycle of data adding new vector reading them updating existing removing outdated vector

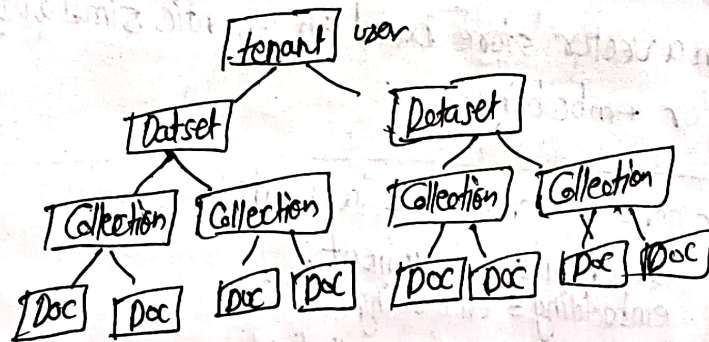
Vector store in Langchain

- Supported stores :- Langchain integrates with multiple vector store (FAISS, Pinecone etc) giving you flexibility in scale features and deployment.

- Common Interface :- A uniform vector store API lets you swap out one backend (FAISS) for another (Pinecone) with minimal code changes.

- Metadata Handling :- Most vector stores in Langchain allow you to attach metadata (e.g time stamps, author) to each document enabling filter-based retrieval.

Chroma vector store :- Chroma is a light weight, open source vector dataset that is especially friendly for local development and small to medium-scale production needs



vector_store = chroma
embedding_function = OpenAIEmbeddings(),
persist_directory = 'my-chroma-db',
collection_name = 'sample'
)

Retrievers

A retriever is a component in Langchain that fetches relevant documents from a data source in response to user query.

- There are multiple type of retrievers.
- All retrievers in Langchain are runables.

Wikipedia Retriever - A wikipedia retriever that queries the wikipedia API to fetch relevant content from a given query.

```
WikipediaRetriever(top_k=result=2, lang='en')
```

Vectorstore Retriever - A vectorstore Retriever in Langchain is the most common type of retriever that lets you search and fetch document from a vector store based on semantic similarity using vector embeddings.

```
vectorstore = Chroma.from_documents(  
    documents=document,  
    embedding=embedding_model,  
    collection_name='my_collection'  
)  
retriever = vectorstore.as_retriever(search_kwargs={'k': 2})
```

Maximal Marginal Relevance (MMR) - MMR is an information retrieval algorithm designed to reduce redundancy in the retrieved result while maintaining high relevance to the query.

```
vectorstore = vectorstore  
vectorstore = FAISS.from_documents(  
    document=docs,  
    embedding=embedding_model  
)  
retriever = vectorstore.as_retriever(  
    search_type='mmr',  
    search_kwargs={'k': 3, 'lambda_mult': 1})
```

... ~~Retriever~~ - Embedding -> Vectorstore -> Retriever -> LLM -> Result

Tools

A Tool is just a python function (or API) that is packed in a way the LLM can understand and call when needed.

LLM (like GPT) are great at:

- Reasoning
- Language generation.

But they cannot do things like:

- Access live data
- Do reliable math
- Call APIs
- Run code

① Build in Tool

② Custom Tools

Agent:- An AI agent is a LLM powered system that can autonomously think, decide and take actions using external tools or API to achieve the goal.

$$\boxed{\text{Agent} = \text{LLM} + \text{Tools}}$$

Built in Tools:- A built in tool is a tool that Langchain already provides for you - it's pre build production ready and requires minimal or no setup.

- You don't have to write function logic yourself - you just import and use it.

→ DuckDuckGo Search Run
Wikipedia Query Run
Python REPL Tool
Shell Tool
Request Get Tool

Custom Tools:- A custom tool is a tool that you define yourself.

- you want to call your own APIs
- you want to encapsulate business tools
- you want the LLM to interact with your database product or app

step 1 create a function
def multiply(a,b):
 """multiply two numbers"""
 return a*b

Step 2 add type hints

```
def multiply (a: int, b: int) -> int:
```

```
    "multiply two number" -> docstring  
    return a*b
```

Step 3 add tool decorator

```
@tool
```

```
def multiply (a: int, b: int) -> int:
```

```
    "multiply two number"  
    return a*b
```

Structured Tool -> A structured tool in Langchain is a special type of tool where the input to the tool follows a structured schema, typically defined using a pydantic model.

```
class MultiplyTool(BaseTool):
```

```
    name: str = 'multiply'
```

```
    description: str = 'multiply two numbers'
```

```
    arg_schema: Type[BaseModel] = multiply_input
```

```
    def _run(self, a: int, b: int) -> int:
```

```
        return a*b
```

Toolkits -> A toolkit is just a collection (bundle) of related tools that serve a common purpose - packaged together for convenience and reusability.

In Langchain:

- A toolkit might be: Google Drive Toolkit
- And it can contain the following tools.

Tool Binding -> Tool Binding is the step where you register tools with a Language Model (LLM) so that:

- 1: The LLM knows what tools are available
- 2: It knows what each tool does
- 3: It knows what input format to use.

Tool Calling - Tool calling is the process where the LLM (language model) decides during a conversation or task that it needs to use a specific tool function - and generates a structured output with:

- the name of the tool
- and the arguments to call it with

★ LLM does not actually run the tool it just suggests the tool and the input arguments. The actual execution is handled by Langchain or you.

Tool Execution - Tool Execution is the step where the actual Python function (tool) is run using the input arguments that the LLM suggested during tool calling.

Tools = Python toolkit
LLM execution = LLM to find (tools)

Ex python toolkit, SQL toolkit it has tools of python and SQL

ReAct

ReAct is a design pattern used in AI agents that stands for Reasoning + Acting. It allows a language model (LLM) to interleave internal reasoning (through) with external action (like tools use) in a structured, multi-step process.

- Instead of generating an answer in one go, the model thinks step by step
- deciding what it needs to do next and optionally calling tools (APIs, calculator, web search etc) to help it.

ReAct is useful for:

- multi-step problems
- Tool-augmented task (web search, database lookup)
- Making the agents reasoning transparent and auditable

Agent & Agent Executor

AgentExecutor orchestrates the entire loop.

- ① Sends inputs and previous messages to the agent
 - ② Gets the next action from agent
 - ③ Executes the tool with provided input
 - ④ Adds the tool's observation back into the history
- ... again with updated history until the agent says Final answer.

Creating an Agent - 2

```
agent = create_react_agent(
```

```
llm=llm,
```

```
tools=[search_tool],
```

```
prompt=prompt
```

```
)
```

Creating an Agent Executor

```
agent_executor = AgentExecutor(
```

```
agent=agent,
```

```
tools=[search_tool]
```

```
verbose=True
```

```
)
```

LANG. GRAPH

Generative AI :- Generative AI refers to a class of artificial intelligence models that can create new content - such as text, images, audio, code or video that resembles human created data.

- LLM based app like chat GPT
- Diffusion model for images
- Code generating LLM like CodeLlama.

Application Areas

- Creative and Business Writing.
- Software Development
- Customer support
- Education.
- Designing

Agentic AI :- Agentic AI is a type of ^{AI} that can take up a task or goal from a user and then work toward completing it on goal from minimal human guidance. It plans, takes action, adapts to changes and seek help only when necessary.

Key characteristics of Agentic AI

- ① autonomous
- ② Goal oriented
- ③ planning
- ④ Reasoning
- ⑤ Adaptability
- ⑥ Context Awareness

Autonomy :- Autonomy refers to the AI system's ability to make decisions and take actions on its own to achieve a given goal, without needing step by step human instructions.

- autonomy in multiple facets
 - ① execution
 - ② Decision making
 - ③ Tool usage
- Autonomy can be controlled
 - ① Permission scope
 - ② Human in the loop
 - ③ Override control
 - ④ Guardrails / policies.

- Autonomy can be dangerous
 - Send out job offers with incorrect salaries.
 - Spending extra money on LinkedIn ads.

② Goal oriented - Being goal oriented means that the AI system operates with a persistent objective in mind and continuously directs its actions to achieve the objective rather than just responding to isolated prompts.

- Goals act as a compass for autonomy
- Goals can come with constraints
- Goals are stored in core memory.
- Goals can be altered.

③ Planning Planning is the agent's ability to break down a high level goal into a structured sequence of actions or subgoals and decide the best path to achieve the desired outcome.

Step 1: Generating multiple candidate plans

~~plan~~
Step 2: Evaluate each plan

Step 3: select the best plan with help of:

- Human in the loop.

④ Reasoning - Reasoning is the cognitive process through which an agent's AI system interprets information, draws conclusions and makes decisions both while planning ahead and while execution actions in real time.

Reasoning During Planning -

- ① Goal decomposition
- ② Tool selection
- ③ Resources estimation

Reasoning during execution

- ① Decision making
- ② HTL handling
- ③ Error handling.

⑤ Adaptability - Adaptability is the agent's ability to modify its plans, strategies or actions in response to unexpected conditions all while staying aligned with the goal.

- ① Failures
- ② External feedback
- ③ Changing goals.

~~Context awareness~~

Context awareness - Context awareness is the agent's ability to understand, retain and utilize relevant information from the ongoing task, past interactions, user preferences, and environmental cues to make better decisions throughout a multi-step process.

① Type of context

- The original goal
- Environment state
- Tool responses

② Context awareness is implemented through memory

③ Short term memory

④ Long term memory.

Components

- brain
- orchestrator
- Tools
- memory
- supervisor

What is Langgraph - ?

- Langgraph is an orchestration framework for building intelligent, stateful and multi-step LLM workflows.
- It enables advanced features like parallelism, loops, branching, memory and resumability - making it ideal for agentic and production grade AI application.
- It models your logic as a graph of nodes and edges instead of a linear chain.

LLM workflows

- ① LLM workflows are a step by step process which we build complex LLM app.
- ② Each step in workflow performs a distinct task - such as prompting, reasoning, tool calling, memory access or decision making

Graph, Nodes and Edges

The system generates an essay topic, collects the student's submission, and evaluates it in parallel on depth of analysis, language quality and clarity of thought. Based on the combined score, it either gives feedback for improvement or approves the essay.

- ① Generate Topic
- ② Collect Essay
- ③ Evaluate Essay

- ④ Aggregate Result
- ⑤ Conditional Routing.
- ⑥ Give feedback
- ⑦ Show success

State

In Langgraph, state is the shared memory that flows through your workflow - It holds all the data being passed between nodes as you graph runs.

- ① accessible
- ② mutable

Reducer

Reducer in Langgraph defines how updates from nodes are applied to the shared state.

Each key in the state can have its own reducer which determines whether new data replace, merge or adds the existing value.

Langgraph Execution model

- ① ~~Define~~ Graph Definition (define - nodes, edges, state schema)
- ② Compilation - (to check graph structure & prepares it for execution)
- ③ Invocation - (Langgraph sends the initial state as a msg to the entry node)
- ④ Super-step Begin - (Execution proceeds in rounds)

What is Persistence

Persistence in Langgraph refers to the ability to save and restore the state of a workflow over time.

What is streaming

In LLM streaming means the model starts sending tokens (words) as soon as they're generated instead of waiting for the entire response to be ready before returning it.

Why streaming

- ① Fast response time - low drop of rates
- ② Mimic human like conversation (Build trust, feels alive, and keeps the user engaged)
- ③ Important for multi-modal UIs
- ④ Better UX for long output such as code
- ⑤ You can cancel midway saving tokens
- ⑥ You can interleave UI updates e.g. show thinking show tool result.

Generators In python, a generator is a special type of iterator that allows you to generate value on the fly, one at a time using the yield keyword instead of return.

LangSmith

What does LangSmith Trace

- Input & output
- All the intermediate step
- Latency
- Cost
- Error
- Tag
- Meta data
- Feedback.

Core concepts of LangSmith.

- ① Project —: whole application called project
- ② Trace —: one single execution is Trace
- ③ Run —: On every Trace run the function called run.

Code Examples

RAG apps have two big failure modes

- Retriever error —: wrong / irrelevant docs retrieved
- Generator error —: model hallucinates or misuses content

In production it's often unclear where the failure happened. Was the retriever bad or did the LLM go on the docs?

- LangSmith automatically records

- User query
- Retrieved documents
- LLM prompt (with inserted docs)
- LLM response

- Every graph execution can be logged in LangSmith as a trace
- Each node (ex. retriever, LLM, tool...) becomes a run inside the trace
- You can visualize the path taken.
- START → Retriever → Reranker → LLM answer → END
- If a workflow branches (conditional / parallel / subgraph), LangSmith captures which path was executed.

MCQ Primitive - standard operations

- Tools
 - tools/list -> Clint ask the server 'what tool do you provide?'
 - tools/call -> Clint tells the server 'plz run this tool with these arguments?'

- Resources
 - resources/list
 - resources/read
 - resources/subscribe/unsubscribe

- Prompts
 - prompts/list
 - prompts/get

ToolNode -> In Langgraph a ToolNode is a prebuilt node type that act as a bridge b/w your graph and external tool (function, APIs)

- A Tool node is a ready made tool that knows how to handle a list of Lanchain tools

Tool Condition -> Tool Condition is a prebuilt conditional edge function that help you graph decide: should the flow go to the ToolNode, or back to the LLM.